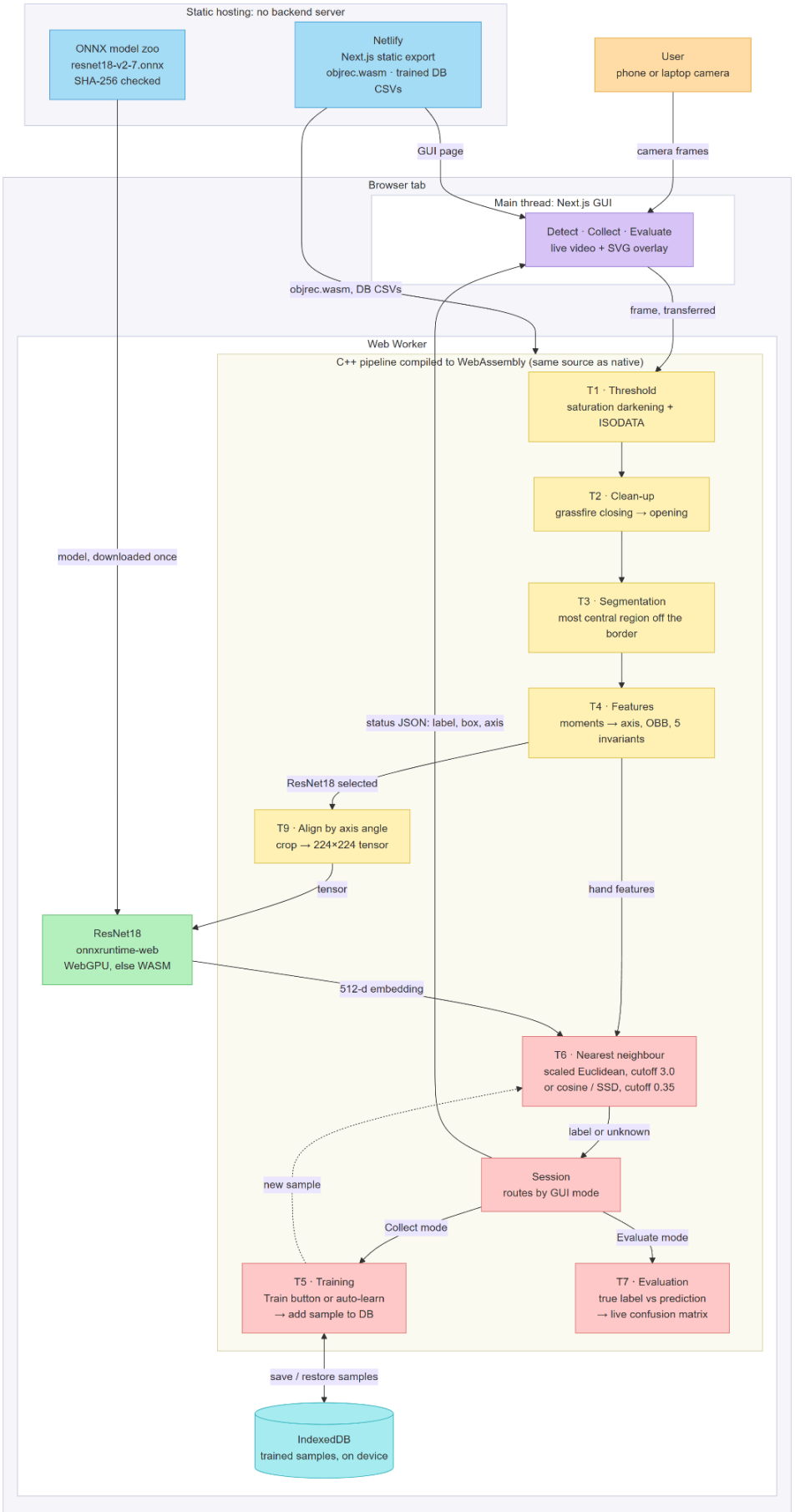

Static hosting: no backend server
ONNX model zoo
resnet18-v2-7.onnx
SHA-256 checked
Netlify
Next.js static export
objrec.wasm · trained DB
CSVs
User
phone or laptop camera
GUI page
camera frames
Browser tab
Main thread: Next.js GUI
Detect · Collect · Evaluate
live video + SVG overlay
objrec.wasm, DB CSVs
frame, transferred
Web Worker
C++ pipeline compiled to WebAssembly (same source as native)
T1 · Threshold
saturation darkening +
ISODATA
T2 · Clean-up
grassfire closing → opening
model, downloaded once
T3 · Segmentation
most central region off the
border
T4 · Features
moments → axis, OBB, 5
invariants
status JSON: label, box, axis
ResNet18 selected
T9 · Align by axis angle
crop → 224×224 tensor
tensor
hand features
ResNet18
onnxruntime-web
WebGPU, else WASM
512-d embedding
T6 · Nearest neighbour
scaled Euclidean, cutoff 3.0
or cosine / SSD, cutoff 0.35
label or unknown
Session
routes by GUI mode
new sample
Collect mode
Evaluate mode
T5 · Training
Train button or auto-learn
→ add sample to DB
T7 · Evaluation
true label vs prediction
→ live confusion matrix
save / restore samples
IndexedDB
trained samples, on device

Sai Supriya Kotturu
FALL-26

1. Project Description:

This project builds an end-to-end processing system to recognise 2D objects on a plain (white) background. The live feed is taken from the browser's camera and is processed in C++. The image is first thresholded after converting it to a single-channel image with saturation darkening, so coloured objects also turn dark. This converts it into a binary image, marking the object as 255 (white) while the white background is marked as 0 (black). A cleanup step then runs, using the grassfire transform: a closing (grow, then shrink) fills holes inside the object, and an opening (shrink, then grow) removes specks. Once the thresholded image is cleaned, segmentation labels the connected regions in the image. The regions touching the boundary of the image are not considered, and the most central remaining region is taken as the object. Its moments give the primary axis and the oriented bounding box, five shape features are computed, and the object is labelled by its nearest neighbour in the trained database; a second classifier compares ResNet18 embeddings instead.

2. Task 1: Threshold

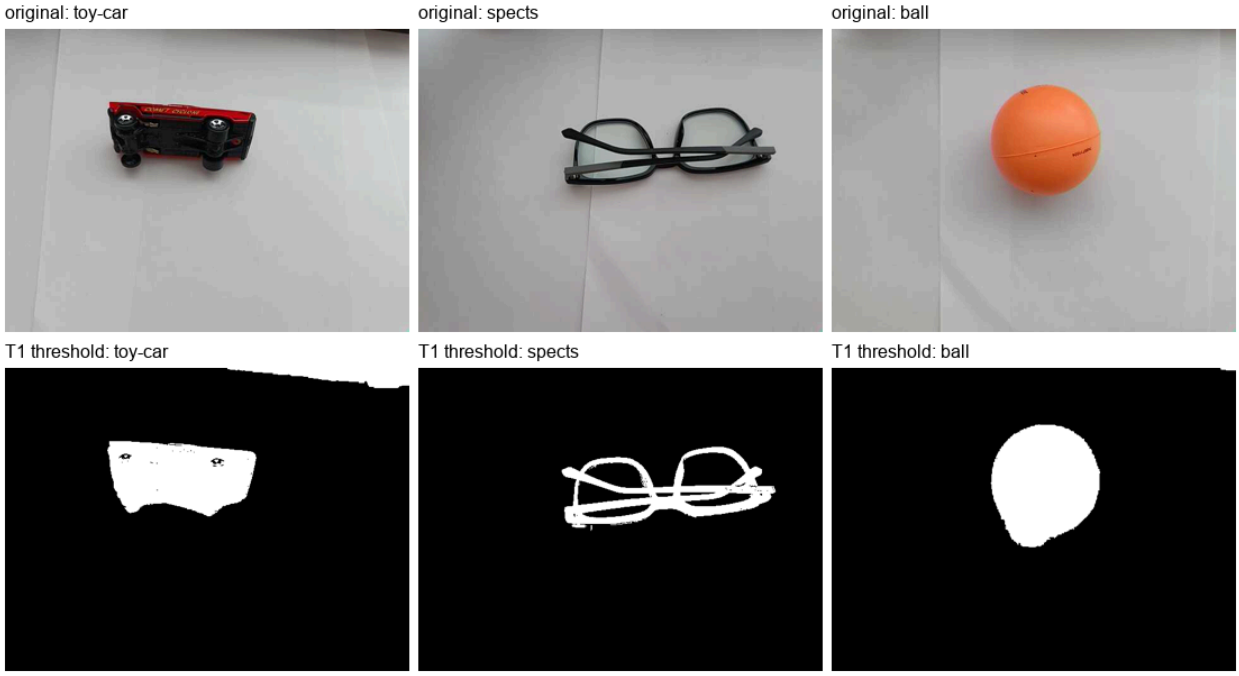
I've handwritten this part: I got the algorithm right in comments. Turning it into C++ took many hints. Writing code in C++ was challenging for my TypeScript brain.

My initial version had a valley-finding approach, which would look for the lowest point between the histogram's two peaks, which was my first instinct, since I remember it from class. But noise in a real histogram creates many small dips, so the valley is unreliable, and the task.pdf named a specific algorithm. The agent pointed this out earlier, so I didn't end up implementing it.

My next version had two sets to track cluster pixels in the image. After every iteration, the cluster centers would update to the mean of the cluster sets. But the agent hinted that using two variables (sum and count) would be efficient since we don't need to track these cluster values, and it takes up more memory for something that gets discarded every iteration.

ISODATA is a pretty simple algorithm, where we track two different sums and counts representing two different clusters. Before we start, we need to have the initial cluster centers, i.e mean values assigned. Since there are two clusters, light pixels (L) and dark pixels (D), we initialize L=255/2 and D = 0. We then loop through a sample of the pixels: every 4th row and column, so 1/16 of the image. Check the absolute difference between the current pixel value and the means. If the current pixel is closer to L (i.e the absolute difference between the current pixel and the average L is less than the other difference), that pixel gets added to sumL (total value of cluster L) and gets counted under that cluster (countL++). Else, it goes to the dark cluster, sumD and countD. After going through the sampled pixels, the means of the two clusters are computed and reassigned. Every iteration

starts with a fresh count and sum. This goes on until both means move by less than the epsilon value (0.5) between iterations, or 15 iterations have run. At the end, we have two cluster centers, and the threshold is the average of these two values which gets returned at the end. Pixels at or below the threshold become the object (255) and the rest become background (0). Before ISODATA, the frame goes through saturation darkening, value × (1 − saturation), so coloured objects like the orange ball also come out dark.
Figure 1 (below): the three report images (top row) and their ISODATA thresholds (bottom row). Thresholds found: toy-car 113, spects 101, ball 120.



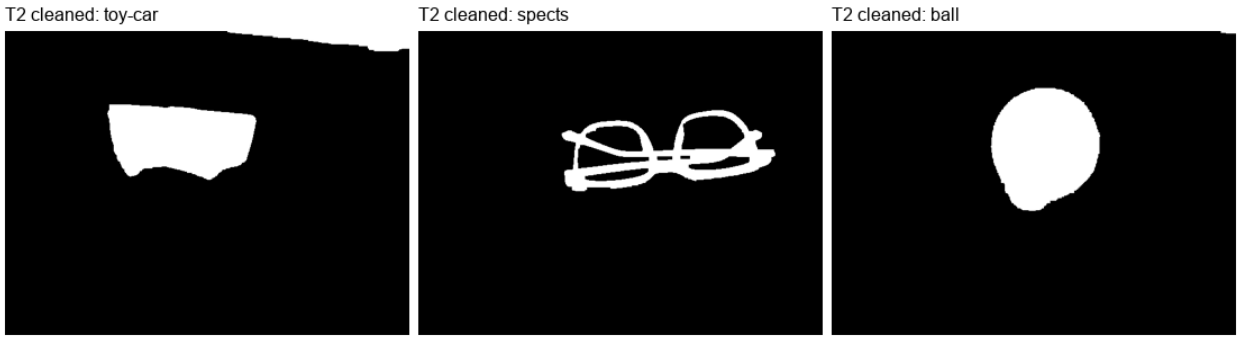
3. Task 2: Cleanup

The thresholded images show two kinds of defects: small specks where paper texture or shadows crossed the threshold, and holes or breaks inside objects where glare or printed text read as background. The clean-up therefore runs a closing followed by an opening, both with 8-connectivity:

- Closing (grow by 2, then shrink by 2) fills holes and narrow gaps inside the object.
- Opening (shrink by 1, then grow by 1) removes specks and thin spurs, including any that the closing joined to the object.

The closing runs first so that no shrink step widens a gap before it has been filled.
Grow and shrink are implemented from scratch with a two-pass grassfire distance transform. The forward pass (top-left to bottom-right) sets each foreground pixel to one more than the smallest distance among its already-visited neighbours; the backward pass repeats this from the opposite corner. Shrink by n removes every foreground pixel whose distance to the background is at most n; grow by n runs the same transform on the inverted image. Pixels outside the frame count as far away, so an object touching the edge

stays touching it. The grassfire code was written by the AI from my pseudocode, and I checked the transform by hand on small grids.
Figure 2 (below) shows the effect: the spectacles' broken rim is joined and the specks are gone. One limitation remains: a 6–8 px glare gap on a metal blade would need $n \geq 4$ to close, which would also merge nearby parts and round off corners, so $n = 2$ was kept.



4. Task 3: Segmentation

Segmentation uses OpenCV's connectedComponentsWithStats on the cleaned image (this stage is not written from scratch). Regions smaller than 0.2% of the frame are discarded, and the five largest of the rest are kept. The primary region is the one whose centroid is closest to the image centre among the regions that do not touch the image border: an object cut off by the frame edge would give misleading features, so it is never used.
Figure 3 (below) colours each region. In the toy-car image a second region, the dark strip along the top edge, touches the border and is excluded.



5. Task 4: Features

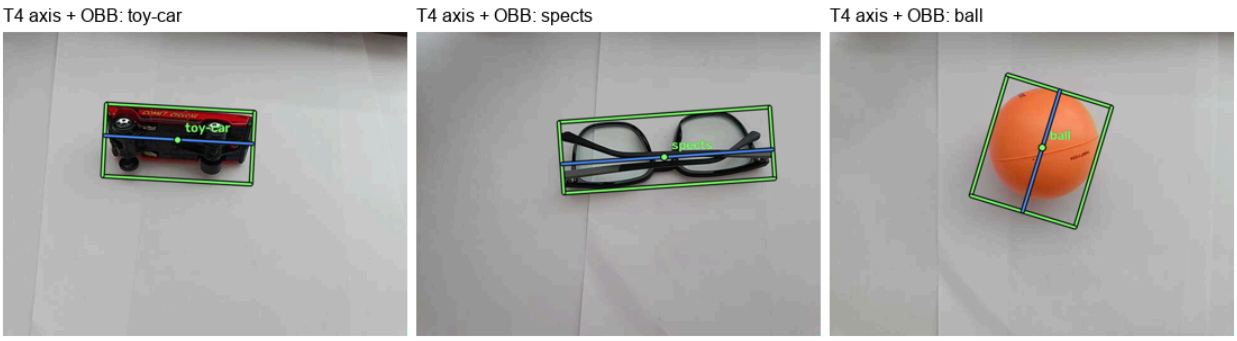
Features are computed for the primary region from its moments, calculated from scratch over the region's pixels. The centroid and the second-order central moments $\mu20$, $\mu02$ and $\mu11$ give the axis of least central moment, $\theta = \frac{1}{2} \cdot \text{atan2}(2\mu11, \mu20 - \mu02)$. Projecting every region pixel onto this axis and its perpendicular gives the minimum and maximum extent along each, which define the oriented bounding box (OBB). The OBB is built from these extents rather than with minAreaRect, so it always agrees with the primary axis. Figure 4 (below) draws the OBB in lime and the primary axis in blue.
The feature vector has five values, each invariant to translation, scale and rotation:

- percentFilled: region area divided by OBB area.
- aspectRatio: the OBB's short side divided by its long side.
- hu1, hu2, hu3: the first three Hu moment invariants, built from the normalised central moments η. hu1 = η20 + η02 measures how spread out the mass is around the centroid, hu2 = (η20 − η02)² + 4η11² measures elongation, and hu3 uses third-order moments to measure asymmetry.

Table 1: feature vectors of the three report images.

image	percentFilled	aspectRatio	hu1	hu2	hu3
toy-car	0.797	0.485	0.221	0.0208	0.000196
spects	0.407	0.339	0.643	0.285	0.000326
ball	0.730	0.867	0.162	0.000447	0.0000586

Measured invariance (extension): each of the 48 test objects was cut out, rotated from 0 to 345° and scaled from 0.5 to 1.5×, and run through the pipeline again. The spread of each feature (standard deviation over mean) stayed under 1% for percentFilled, aspectRatio and hu1, at 2–3% for hu2 and 3.5–5% for hu3 (Figure 9).



6. Task 5: Training

Training adds labelled feature vectors to a database stored as a CSV file, one row per sample: the label followed by the five features. A second CSV, kept separately for ResNet18, stores each sample's 512-value embedding, since embeddings from different models can't be compared. Samples can be added in three ways, all through the same pipeline:

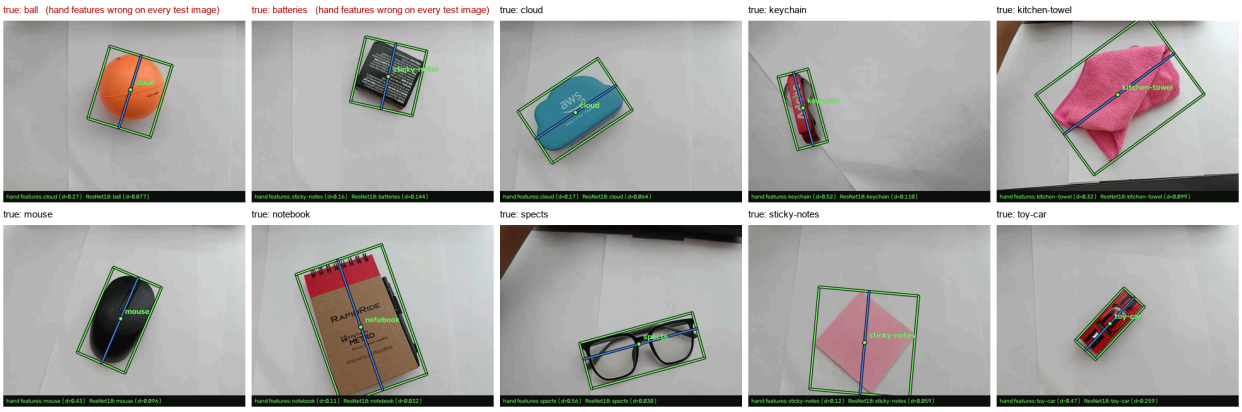
- In the live program: press n, type the label into the video window, press Enter.
- In the web GUI's Collect mode: set a label and press Train, or let auto-learn collect samples (see Extensions).
- In batch, with the train program, which reads data/objects/<label>/ folders and uses each folder name as the label.

The database used for all results was built in batch from 58 captures of the 10 objects, 4 to 7 per object, at different positions and orientations.

7. Task 6: Classification

A new object is classified by nearest neighbour against the database, using the scaled Euclidean distance $d = \sqrt{\Sigma ((x_i - y_i) / \sigma_i)^2}$, where σ_i is the standard deviation of feature i over the database. σ_i is floored at 1% of the feature's mean magnitude, so a feature that barely varies in the database can't dominate the distance. If the nearest distance is above a cutoff (3.0), the object is reported as unknown (extension).

Figure 5 (below) shows one held-out test image per class with the label predicted from the hand features; the caption under each image also gives the ResNet18 prediction for comparison. Eight of the ten are classified correctly. For ball and batteries the hand features are wrong on every test image, so a failure is shown for those two; Task 7 explains why.



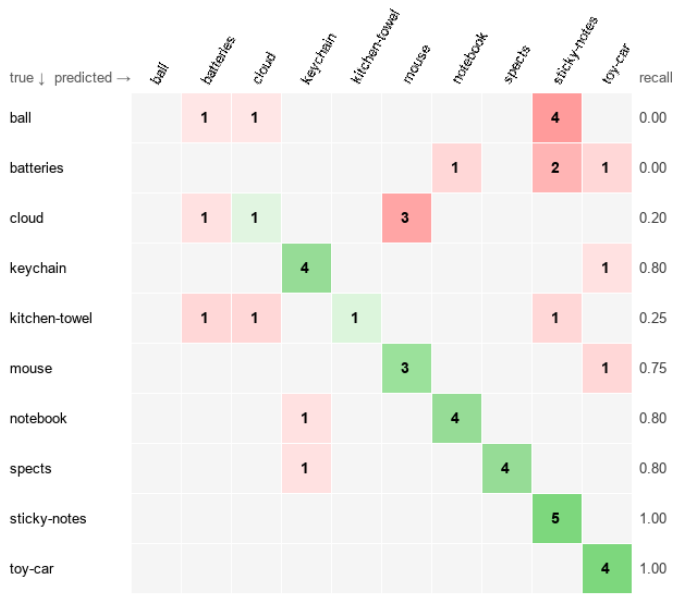
8. Task 7: Evaluation

The evaluation uses a separate test set: the database holds all 58 training captures, and 48 further captures of the same objects, taken at other positions and orientations, are classified. One notebook capture yields no region, so 47 are scored. The eval program tallies each prediction against the true label (the folder name); in the GUI, Evaluate mode does the same live, with the user setting the true label and recording each prediction.

Hand features classify 26 of the 47 test images correctly (55%). The task asks for a 5×5 confusion matrix; with 10 objects, the full 10×10 matrix is shown in Figure 6 (below; rows are the true label, columns the prediction).

Objects with a distinctive silhouette are recognised well: sticky-notes 5/5, toy-car 4/4, keychain, notebook and spects 4/5, mouse 3/4. Blob-like objects are not: ball 0/6 (4 called sticky-notes), batteries 0/4, cloud 1/5 (3 called mouse) and kitchen-towel 1/4. A disc and a square both lack a well-defined primary axis, so their oriented boxes, aspect ratios and Hu moments come out nearly the same. Seven of the ten objects are blob-like (hu1 between 0.17 and 0.20), and that is where adding objects beyond the required five hurt the classical method most.

T7: hand features, scaled Euclidean NN: 26/47 (55%)



9. **Task 8: Demo**

   Demo:
   https://github.com/supriya-kotturu/real-time-2d-object-recognition/blob/main/public/demo/samples.gif
   Live demo: https://real-time-2d-object-recognition.netlify.app.

10. **Task 9: One-shot classification with ResNet18 embeddings**

    The embedding classifier reuses the segmentation and features from Tasks 1–4 and follows the instructor's utilities, adapted and ported to OpenCV 5:

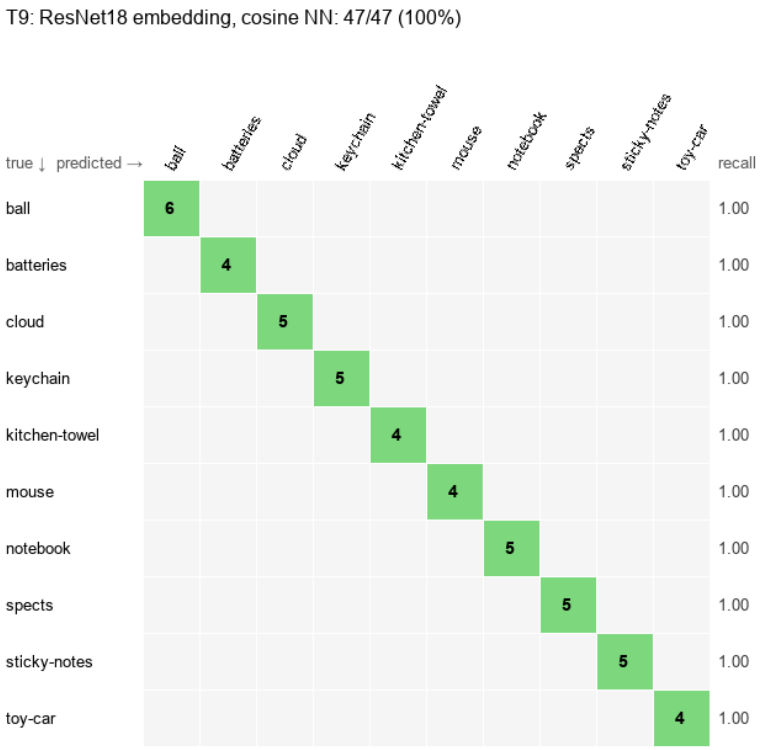
    - Step A: from Task 4, the centroid, the primary-axis angle θ and the region's extents along both axes.
    - Step B: rotate the original colour frame by θ about the centroid, so the primary axis lies along x. The axis has a 180° ambiguity; it is resolved with the sign of the third moment along the axis, so an object turned upside down gives the same crop.
    - Step C: crop the now axis-aligned bounding box (Figure 7, below).
    - Step D: resize to 224×224 and normalise with the ImageNet means (124, 116, 104) and a standard deviation of 0.226.
    - Step E: run ResNet18 v2 from the ONNX model zoo and take the 512-value output of its global-average-pooling layer as the embedding.

    Classification is one-shot nearest neighbour with cosine distance (1 − cosine similarity); the sum of squared differences (SSD) is available as an alternative. Rotation invariance comes from the alignment in step B, not from the network.
    Table 2: accuracy of each method on the same data.

protocol	hand features	ResNet18 cosine	ResNet18 SSD
Task 7 split (58 training, 47 test)	26/47 (55%)	47/47 (100%)	45/47 (96%)
one-shot (1 training image per object)	22/47 (47%)	44/47 (94%)	42/47 (89%)
leave-one-out (58 training images)	32/58 (55%)	57/58 (98%)	57/58 (98%)

The embedding classifies every test image correctly (Figure 8, below) and still scores 44 of 47 with a single example per object, against 26 and 22 for the hand features. It sees colour and texture, which the shape features ignore by design: an orange ball and a pink square of sticky notes have nearly the same shape features but very different embeddings. Cosine distance does better than SSD because it ignores the embedding's overall magnitude, which changes with image contrast.
Limitation: the embedding is sensitive to exposure. A uniform brightening of 10 grey levels turned the hardest toy-car test image (cosine distance 0.26) into keychain, while the thresholded shape features barely change.



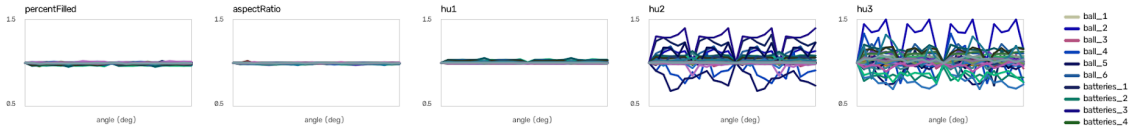


11. Extensions

I've created a website which runs the whole system in the browser (the last item below). The extensions I implemented:

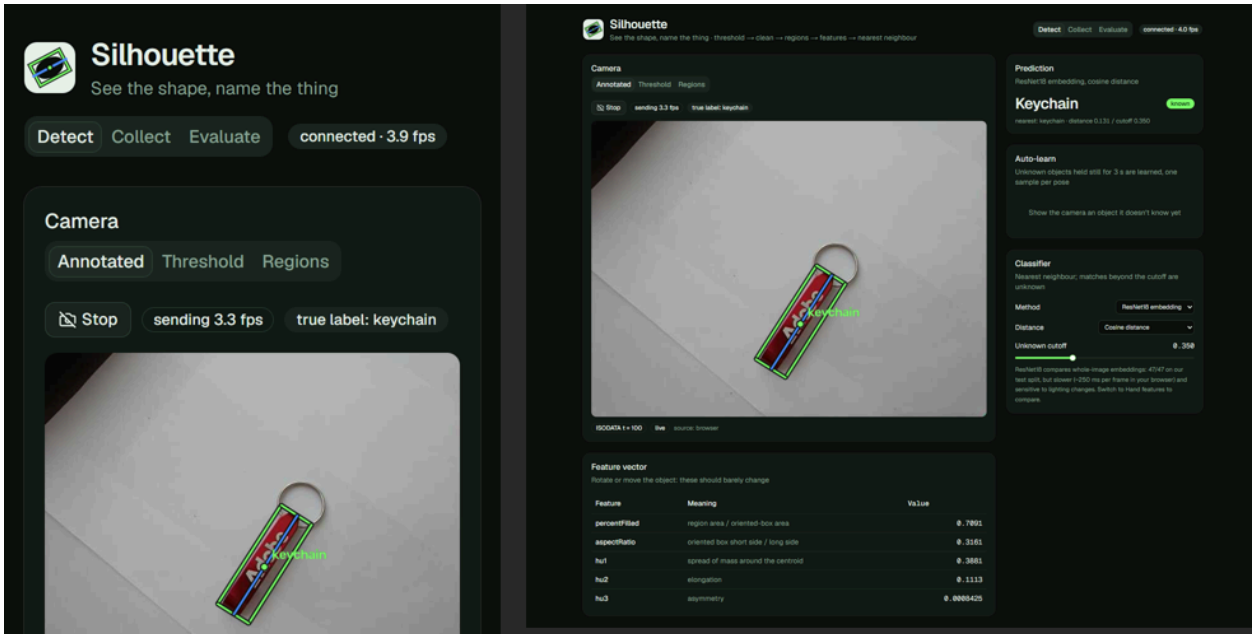
- More than one stage from scratch: the ISODATA threshold (Task 1) and grassfire grow and shrink (Task 2); the moments and features are also computed without OpenCV's moments function.
- Five more objects than required (10 in total); the impact on accuracy is discussed under Task 7.
- Unknown-object detection: a nearest distance above a per-metric cutoff is reported as unknown.
- Auto-learn: an unknown object held still for 3 s triggers a "name this object" prompt; the system then collects 5 samples, one per pose, where a new pose means a move of 10% of the image diagonal or a 20° rotation, followed by 0.6 s of stillness.
- A better GUI: a web app with Detect, Collect and Evaluate modes, a live confusion matrix, a switch between hand features and ResNet18 (cosine or SSD), and a gallery of trained samples (Figure 10).
- Other metrics and embeddings: cosine distance and SSD for the ResNet18 embedding, compared with the hand features in Table 2.
- Measured invariance: feature drift under rotation and scale (Task 4, Figure 9).
- Deployment with no server: the same C++ pipeline compiled to WebAssembly runs in a Web Worker in the browser and gives identical results to the native build, and

ResNet18 runs in onnxruntime-web on the GPU through WebGPU (about 15 fps on a laptop). It works on phones with the back camera, and trained samples are stored on the device. After one-shot training it recognised a phone lying on a patterned bedsheet.



Further improvements

Robust unknown detection: Auto-learn currently requires every frame in the 3 s window to be classified unknown; a single frame matching a trained object resets the timer. Objects whose silhouette resembles a trained one sit near the cutoff and flicker between a trained label and unknown, so they are rarely learned (see the look-alikes demo in Task 8). Lowering the cutoff, as in the auto-learn demo, hides this but increases false unknowns on trained objects. A sliding-window vote (learn when at least 80% of the frames in the window are unknown), or hysteresis on the distance, would tolerate per-frame noise without moving the decision boundary.


Silhouette
See the shape, name the thing
Detect Collect Evaluate
connected · 3.9 fps
Camera
Annotated Threshold Regions
Stop
sending 3.3 fps
true label: keychain
ISODATA t = 100
live
source: browser
Prediction
ResNet18 embedding, cosine distance
Keychain
known
nearest: keychain · distance 0.131 / cutoff 0.350
Auto-learn
Unknown objects held still for 3 s are learned, one sample per pose
Show the camera an object it doesn't know yet
Classifier
Nearest neighbour; matches beyond the cutoff are unknown
Method
ResNet18 embedding
Distance
Cosine distance
Unknown cutoff
0.350
ResNet18 compares whole-image embeddings: 47/47 on our test split, but slower (~250 ms per frame in your browser) and sensitive to lighting changes. Switch to Hand features to compare.
Feature vector
Rotate or move the object: these should barely change
Feature
Value
percentFilled
0.7091

12. Reflection

Through this project, I learned that object recognition fundamentally boils down to a distance-matching algorithm: it measures how closely an input's feature vector matches a known vector stored in the database. Because of this, it can only ever recognize things it has explicitly been trained on.

We have to be very precise about what the algorithm actually "knows." Right now, it's just working with simple shape features. If our database has a circular object labeled "ball," any circular object it encounters will get classified the exact same way. Whether it's an actual ball, the top-down view of a can, or a jar of O'Keeffe's hand cream.

This project also taught me all the unexpected hurdles involved in deploying an application to a static server. Naturally, the first recommendation AI spits out is to set up a backend server to listen for incoming video frames via a WebSocket connection. But hosting a server like that comes with a price tag. I tried deploying one to Hugging Face, only to discover after the fact that hosting requires a paid PRO plan. I didn't have an AWS account and couldn't afford to attach a debit card to a cloud provider, which quickly made me realize just how broke I am.

Instead of giving up, I pivoted: I decided to run everything locally in a web worker. I sparred with the AI on which transport mechanism to pick (polling vs. WebSocket vs. alternatives) and weighed the pros and cons of each. While the initial suggestions were all server-based, I pushed for running the entire pipeline inside a web worker instead. After testing and polishing, we ended up with a fully responsive web application: users can train new objects using their phone's back camera, all samples are stored locally on their own device, and it successfully recognizes objects on a light surface, completely without a server.

13. Acknowledgement

Materials and code used:

- Bruce A. Maxwell's course utilities: the Task 9 alignment and ResNet18 embedding approach, adapted to OpenCV 5 (the original is kept unmodified in third_party/maxwell).
- ResNet18 v2 from the ONNX model zoo; OpenCV 5; cpp-httplib; onnxruntime-web; Emscripten; Next.js and shadcn/ui.

AI use. I used Claude (Anthropic, through Claude Code in VS Code) across three sessions; the conversations are in the transcripts/ folder of the repository. I set it up as a staff C++ and computer-vision mentor with explicit rules: every hint was logged, a repeated hint cost me a point and a visible improvement earned one (I finished at +13), it gave a progress update every six hours of work, and every design decision was recorded with its options and trade-offs in docs/mentorship/ (decisions D-00 to D-29).

Sai Supriya Kotturu
FALL-26

Task 1 (threshold): I wrote ISODATA and applyThreshold myself. Claude reviewed my pseudocode and code and gave hints on C++ (headers, namespaces, const references, cv::Mat output parameters) and on the algorithm (sampling, a running sum and count instead of cluster sets, the convergence test and the iteration cap).
Task 2 (clean-up): I wrote the grassfire pseudocode and hand-computed small test grids; Claude implemented it, wrote the tests, and asked me to explain the design back (the two passes, border handling).
Tasks 3 to 9 and the tools (train, eval, inspect, invariance): written by Claude from decisions I made, such as using OpenCV for Task 3 to save time for extensions, the train/test protocol, per-pose auto-learn and the choice of ResNet18. I captured all the images, tested on my own objects and reviewed the results.
Web GUI and deployment: I made the architecture calls after Claude laid out the options and trade-offs (browser camera, WebSocket transport, classifier toggle, theme, on-device storage). When server hosting turned out to need a paid plan, I proposed running everything in a web worker with no server, and Claude implemented it.
Debugging: using the threshold and region views, I traced a misclassified phone to segmentation (my hand merged with the phone and touched the border); Claude confirmed the chain.
This report: the project description, the Task 1 narrative and the reflection are mine. Claude drafted the technical descriptions of Tasks 2 to 9 and the extension list from the code and evaluation output, made the figures and tables, and suggested factual corrections to my sections, which I reviewed.